índice

Reto Práctico: Comprobación de Consultas XPath....2
Paso 1: El Archivo micatalogo.xml (Referencia)....2
Paso 2: El Código JavaScript para Comprobar XPath (micatalogo.js)....2
1. Añadir la Función de Comprobación (al inicio de micatalogo.js)....2
2. Implementación (Dentro del fetch existente)....3
Paso 3: Desafíos y Resultados Esperados....4
Paso 4: Ejecución....5

Unidad Didáctica 1.5: Introducción a XPath (XML Path Language)

1. ¿Qué es XPath? (XML Path Language)

XPath no es un lenguaje de programación como JavaScript, sino un **lenguaje de consulta** (o de rutas) diseñado para **seleccionar nodos** (elementos, atributos y textos) dentro de un documento XML o HTML (que internamente también es un árbol DOM).

Su principal función en el desarrollo web y de sistemas es permitirnos buscar información de forma precisa.

Uso principal	Ejemplo
Búsqueda y Filtro	Seleccionar solo los productos que cuestan más de 100 €.
Transformación (XSLT)	Indicar qué datos del XML deben convertirse a HTML o PDF.
Validación (XSD)	Aplicar reglas de validación complejas a ciertas partes del documento.

2. El Modelo de Árbol

Para usar XPath, debemos entender que un documento XML (como nuestro micatalogo.xml) es interpretado como una **estructura de árbol** jerárquica, compuesta por distintos tipos de nodos.

Ejemplo de Jerarquía en micatalogo.xml:

- La **Raíz** es el nodo principal del documento (imaginario).
- El **Elemento Raíz** es <catalogo>.
- Los hijos directos de <catalogo> son los nodos <producto>.
- Los hijos de <producto> son <nombre>, <descripcion>, <categoria>, etc.
- El **Atributo** id del producto es un nodo diferente del elemento.

3. Ejes y Sintaxis Básica de Navegación

Los **Ejes** son los símbolos que definen la dirección de la búsqueda. Son la base para construir cualquier ruta.

Símbolo	Nombre	Descripción	Ejemplo	Resultado
/	Eje Absoluto	Selecciona el nodo hijo directo de otro nodo. La ruta debe empezar en la raíz (/catalogo).	/catalogo/producto	Selecciona todos los <producto> que son hijos directos de <catalogo>.
//	Eje Descendiente	Selecciona nodos que coincidan con el nombre, sin importar su posición o	//nombre	Selecciona **todos** los nodos <nombre> en cualquier parte del

Símbolo	Nombre	Descripción	Ejemplo	Resultado
		profundidad en el árbol.		XML.
@	Eje de Atributo	Selecciona un **atributo** en lugar de un elemento.	//producto/@id	Selecciona los valores de los atributos id de todos los productos.
.	Nodo Actual	Hace referencia al nodo que se está procesando actualmente.	.	Usado en XSLT para obtener el valor del nodo actual.
..	Nodo Padre	Selecciona el nodo padre del nodo actual.	//precio/..	Devuelve el elemento <producto> (el padre de <precio>).

4. El Poder de los Predicados (Filtros)

Los predicados son el mecanismo más potente de XPath. Permiten filtrar los nodos seleccionados en función de su contenido, sus atributos, su posición o el valor de sus hijos. Se encierran entre corchetes [].

La sintaxis es: ruta/eje[predicado]

A. Filtrar por Atributo

Se usa el símbolo @ dentro del predicado para hacer referencia a un atributo.

Desafío	Ruta XPath	Explicación
Producto Único	//producto[@id='AURIo01']	Selecciona el <producto> cuyo atributo id es igual a 'AURIo01'.
Moneda	//precio[@moneda='EUR']	Selecciona los <precio> cuyo atributo moneda tiene el valor 'EUR'.
Atributo Existente	//producto[@id]	Selecciona los <producto> que **tienen** el atributo id (sin importar su valor).

B. Filtrar por Contenido del Nodo

Se compara el valor de un nodo hijo o del nodo actual.

Desafío	Ruta XPath	Explicación
Por Categoría	//producto[categoria='Audio']	Selecciona los <producto> que tienen un hijo <categoria> con el valor **'Audio'**.

Desafío	Ruta XPath	Explicación
Posición	/catalogo/producto[2]	Selecciona el **segundo** <producto> en el catálogo.

C. Operadores Lógicos y Numéricos

Se pueden usar operadores de comparación (>, <, =, !=) y operadores lógicos (and, or, not).

Operador	Significado
=	Igual a
!=	Diferente a
> / <	Mayor que / Menor que
and / or	Y / O

Uso de number() para Comparaciones

Cuando comparamos el contenido de un elemento que es un número (como el <precio>), es fundamental usar la función **number()** para evitar que XPath lo interprete como texto.

Desafío	Ruta XPath	Explicación
Precio Alto	//producto[number(precio) > 100]	Selecciona los <producto> cuyo precio (interpretado como número) es mayor que 100.
Doble Condición	//producto[categoria='Audio' and number(precio) < 100]	Productos de categoría Audio **Y** con precio menor a 100.

---

5. Referencia Rápida: Los Nodos Especiales

Nodo	XPath	Descripción
Elemento	//nombre	La etiqueta completa (<nombre>...)
Atributo	//imagen/@url	El valor del atributo (smartwatch.jpg)
Texto	//nombre/ text()	Solo el contenido de texto dentro del elemento, eliminando etiquetas (ej. 'Auriculares Inalámbricos Pro-Sound').
Raíz	/	Punto de partida del documento.

Reto Práctico: Comprobación de Consultas XPath

Paso 1: El Archivo micatalogo.xml (Referencia)

Usaremos la estructura de su catálogo como referencia:

XML

```
<catalogo>
  <producto id="AURI001">
    <nombre>Auriculares Inalámbricos Pro-Sound</nombre>
    <categoria>Audio</categoria>
    <precio moneda="EUR">99.99</precio>
    <imagen url="auriculares.jpg"/>
  </producto>

  <producto id="SWATCH002">
    <nombre>Smartwatch Fitness X3</nombre>
    <categoria>Wearables</categoria>
    <precio moneda="EUR">79.50</precio>
    <imagen url="smartwatch.jpg"/>
  </producto>

  <producto id="PORT003">
    <nombre>Portátil Ultraligera Air</nombre>
    <categoria>Informática</categoria>
    <precio moneda="EUR">1299.00</precio>
    <imagen url="laptop.jpg"/>
  </producto>
</catalogo>
```

Paso 2: El Código JavaScript para Comprobar XPath (micatalogo.js)

Debe añadir una nueva función al inicio de su micatalogo.js y luego llamar a esta función dentro de la cadena de .then(), donde ya tienen definido el objeto xmlDoc.

1. Añadir la Función de Comprobación (al inicio de micatalogo.js)

Esta función es la clave. Recibe la ruta XPath y el documento XML, y se encarga de recorrer los resultados e imprimirlos en la Consola del navegador.

JavaScript

```
/**
 * Función que ejecuta una consulta XPath y muestra los resultados en la Consola.
 * @param {string} xpathQuery - La ruta XPath que se desea comprobar.
 * @param {Document} xmlDoc - El objeto DOM del XML parseado.
 */
function comprobarXPath(xpathQuery, xmlDoc) {
  console.groupCollapsed(`RESULTADOS DE LA CONSULTA: "${xpathQuery}"`);

  // El método evaluate() de JavaScript ejecuta la consulta XPath
  // XPathResult.ORDERED_NODE_ITERATOR_TYPE es el tipo de resultado para listas de nodos
```

```
  const resultado = xmlDoc.evaluate(
    xpathQuery,
    xmlDoc,
    null,
    XPathResult.ORDERED_NODE_ITERATOR_TYPE,
    null
  );

  let nodo = resultado.iterateNext();
  let contador = 0;

  // Recorrer e imprimir los nodos
  while (nodo) {
    // Comprobamos si es un ATRIBUTO o un ELEMENTO y mostramos su valor.
    const valor = (nodo.nodeType === 2) ? nodo.value : nodo.textContent;
    console.log(`[${contador + 1}]  Tipo: ${nodo.nodeName || 'Valor'} -> Valor: ${valor.trim()}`);
    contador++;
    nodo = resultado.iterateNext();
  }

  if (contador === 0) {
    console.warn('La consulta XPath NO devolvió ningún nodo. Revisa la ruta.');
  } else {
    console.info(`Consulta finalizada. Total de ${contador} nodos encontrados.`);
  }

  console.groupEnd();
}
```

2. Implementación (Dentro del fetch existente)

Pegar las llamadas a comprobarXPath() justo después de haber parseado el XML y definido la variable xmlDoc.

JavaScript

```
// micatalogo.js (Parte de la implementación existente)

fetch('micatalogo.xml')
  .then(response => response.text())
  .then(xmlTexto => {
    // ... (código existente de DOMParser) ...
    const parser = new DOMParser();
    const xmlDoc = parser.parseFromString(xmlTexto, "application/xml");

    // ======================================================
    // A PARTIR DE AQUÍ: LLAMADAS DE PRUEBA DE XPATH
    // ======================================================

    // Desafío 1: Todos los atributos 'url'
    comprobarXPath("//imagen/@url", xmlDoc);

    // Desafío 2: Todos los productos de 'Audio'
    comprobarXPath("//producto[categoria='Audio']", xmlDoc);
```

```
    // Desafío 3: Productos caros
    comprobarXPath("//producto[number(precio) > 100]/nombre", xmlDoc);

    // ... (código existente para generar el catálogo, etc.) ...
    // ...
  })
  // ... (rest of the fetch chain) ...
```

Paso 3: Desafíos y Resultados Esperados

Escribir la solución XPath para cada desafío. Copiar y pegar como el primer parámetro de la función comprobarXPath(RUTA_AQUI, xmlDoc);.

Desafío	Solución XPath	Resultados en Consola (Ejemplo)
1. Atributos: Seleccionar **todos** los valores de los atributos url de las imágenes.	//imagen/@url	[1] Tipo: url -> Valor: auriculares.jpg [2] Tipo: url -> Valor: smartwatch.jpg [3] Tipo: url -> Valor: laptop.jpg
2. Filtrado por Valor: Seleccionar **todos** los nodos <producto> cuya categoría sea exactamente **'Audio'**.	//producto[categoria='Audio']	[1] Tipo: producto -> Valor: (el contenido del nodo Auriculares...)
3. Filtrado Numérico: Seleccionar los **nombres** de los productos cuyo precio sea **mayor de 100** (sin incluir la moneda).	//producto[number(precio) > 100]/nombre	[1] Tipo: nombre -> Valor: Portátil Ultraligera Air
4. Búsqueda Específica por ID: Seleccionar la descripción del producto con id='SWATCH002'.	//producto[@id='SWATCH002']/descripcion	[1] Tipo: descripcion -> Valor: Reloj inteligente con monitorización...
5. Operador Lógico (o): Seleccionar los nombres de los productos que son de categoría 'Audio' **o** 'Informática'.	//producto[categoria='Audio' or categoria='Informática']/nombre	[1] Tipo: nombre -> Valor: Auriculares Inalámbricos Pro-Sound [2] Tipo: nombre -> Valor: Portátil Ultraligera Air

Paso 4: Ejecución

1. Guardar los cambios en micatalogo.js.
2. Abrir micatalogo.html en el navegador (usando Live Server o similar).
3. Abrir la **Consola de Desarrollador** (F12 o Ctrl+Shift+J / Cmd+Option+J).
4. Comprobar que por cada consulta (comprobarXPath) se ha desplegado un bloque de resultados con el listado de nodos encontrados.